Assignment 1: Polars vs Pandas

Performance, memory efficiency and usability of two Python DataFrame libraries

Name: Surya Koundinya C

Batch: G41-AIML

1. Introduction

This report compares **Pandas** and **Polars**, two Python libraries for in-memory tabular data analysis. Both provide a DataFrame abstraction for loading, filtering, aggregating, joining and transforming data. The aim is to measure how they differ in execution time, memory use, scalability and ease of use when applied to the same real-world dataset of roughly 10.9 million rows.

Pandas is the long-established standard for data analysis in Python. It is built around the DataFrame and an explicit row index, executes each operation immediately (eager execution) and runs most operations on a single thread. Its main strength is a very large ecosystem that integrates with libraries such as NumPy, scikit-learn and Matplotlib.

Polars is a newer DataFrame library written in Rust and based on the Apache Arrow columnar memory format. It runs many operations in parallel across CPU cores, uses an expression-based API with no row index, and offers both an eager API and a lazy API in which a query optimizer plans the whole computation before running it. Only the eager API is evaluated in this report. Table 1 summarizes the key differences.

Table 1. Key differences between Pandas and Polars

Aspect	Pandas	Polars
Implementation	Python with C/Cython extensions; NumPy-based (optional Arrow support)	Rust core; Apache Arrow columnar format
Parallelism	Mostly single-threaded	Multi-threaded by default
Execution model	Eager	Eager and lazy (query optimizer)
API style	Row index, bracket indexing, mutable objects	Expressions (`pl.col(...)`), no index, immutable
Ecosystem	Very mature and widely integrated	Newer and growing quickly

2. Environment Setup

All experiments were run on a MacBook Pro using Visual Studio Code and an isolated Python virtual environment created inside the project folder. Both libraries, together with the supporting packages used for measurement and plotting, were installed with pip:

```
python3 -m venv .venv
source .venv/bin/activate
pip install pandas polars pyarrow numpy psutil matplotlib jupyter ipykernel
```

3. Dataset Selection

The dataset is the **NYC Yellow Taxi Trip Records for January 2016**, published by the NYC Taxi & Limousine Commission (TLC) and obtained as a single CSV file from Kaggle. It is large enough to expose real performance differences while still fitting in the memory of a laptop.

Table 3. Dataset description

Property	Value
Source	NYC TLC Yellow Taxi Trip Records, January 2016 (CSV from Kaggle)
File	yellow_tripdata_2016-01.csv
File size	1.71 GB
Number of rows	10,906,858
Number of columns	19
Main columns used	VendorID, tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance, payment_type, fare_amount, tip_amount, total_amount
Second dataset (for joins)	Payment-type lookup table: 6 rows, 2 columns (payment_type, payment_name), built from the TLC data dictionary

The January 2016 file contains pickup and dropoff coordinates rather than zone IDs, so a zone lookup join was not possible. The join was instead performed on `payment_type` against a small dimension table, which is a legitimate key-based join. The raw data contains some rows with zero or negative fares or distances and a passenger count of 0. These rows were kept unchanged in both libraries so that the comparison is fair.

4. Implementation

The same operations were implemented in both libraries and applied to the full dataset. Table 4 describes each operation, and Table 5 shows the corresponding code, which also illustrates the difference in usability between the two APIs.

Table 4. Operations implemented in both libraries

Operation	Description
Load the dataset	Read the CSV file, parsing the two datetime columns
Filter rows	Rows with `trip_distance > 5` and `fare_amount > 20`
Select columns	Select five columns (VendorID, passenger_count, trip_distance, fare_amount, tip_amount)
Group and aggregate	Group by `payment_type`: mean fare, mean tip, total distance, trip count. A second grouping uses the pickup hour (derived from a datetime column) and computes the mean `total_amount`
Join	Left join with the payment-type lookup table on the common key `payment_type`
Sort	Sort all rows by `total_amount`, descending
Transform	Add two new columns: trip duration in minutes and tip percentage

5. Performance Benchmarking

Execution time was measured as follows:

- Time was measured with `time.perf_counter()`, calling `gc.collect()` before every run.
- Each operation was run **7 times** and the **median** is reported (3 runs for CSV loading).
- Each library was benchmarked in its **own Python process**. An initial version that ran both libraries in the same process produced inconsistent Polars timings that were likely caused by memory pressure, so the experiments were redesigned to isolate them.
- Polars columns were consolidated into contiguous memory with `rechunk()` after loading. The effect of this step is shown in Table 6.

Table 6 shows the median execution time of each operation on the full dataset (10,906,858 rows). Speedup is Pandas time divided by Polars time, so values above 1 mean Polars was faster.

Table 5. Median execution time on 10.9 million rows (7 runs; Polars after rechunk)

Operation	Pandas (s)	Polars (s)	Polars speedup
filter	0.1099	0.0432	2.55x
select	0.0004	0.0002	1.61x
groupby	0.3189	0.1482	2.15x
groupby_hour	0.1341	0.1826	0.73x
join	0.1202	0.0534	2.25x
sort	2.9444	1.1002	2.68x
transform	0.0655	0.0515	1.27x

Three notes are needed to read this table correctly:

- **select** takes well under a millisecond in both libraries, which indicates that no data is copied when columns are selected. The 1.6x ratio is therefore not meaningful.
- **filter** for Pandas was the least stable measurement: its full-size time was 0.064 s, 0.110 s and 0.173 s in three separate runs, while Polars stayed near 0.043 s. The filter speedup is best described as roughly 1.4x to 2.5x.
- **Loading the CSV** was measured in a separate three-run experiment: about 5.31 s for Pandas and 4.36 s for Polars (1.22x). Parsing a 1.7 GB text file appears to dominate for both libraries, so the gap is small.

Polars reads CSV files in parallel, and the resulting columns can be stored in many separate chunks. In an earlier run on the frame as returned by `read_csv`, several operations were noticeably slower than after calling `rechunk()`, as shown in Table 6. Each value is from a single run, so differences

below about 30% are within normal run-to-run variation. The time taken by `rechunk()` itself was not included.

Table 6. Polars execution time before and after `rechunk()` (10.9 million rows)

Operation	As loaded (s)	After rechunk (s)
filter	0.0707	0.0432
groupby	0.5633	0.1482
groupby_hour	0.5562	0.1826
join	0.0667	0.0534
sort	1.6086	1.1002
transform	0.0341	0.0515

The clearest effects are on the two groupby operations (about 3.8x and 3.0x faster after rechunking) and on sort. Transform was slightly faster before rechunking, so the effect depends on the operation. Table 6 uses the rechunked frame.

6. Memory Analysis

Memory was measured in a separate script, with one library per process, on the same dataset and workload: load the CSV, run the `payment_type` groupby, and sort by `total_amount`. Two values were recorded: the size of the loaded DataFrame (`memory_usage(deep=True)` in Pandas, `estimated_size()` in Polars) and the **peak resident set size (RSS)** of the process from `resource.getrusage`. Each library was measured once.

Table 7. Memory use while loading the CSV, running a groupby and sorting (single run each)

	Pandas	Polars
In-memory DataFrame size	1,669 MB	1,581 MB
Peak process memory (RSS)	2,186 MB	2,811 MB

The loaded DataFrame is about 5% smaller in Polars, which is a small difference. Peak memory was about 29% higher for Polars (2,811 MB vs 2,186 MB). This does not match the common expectation that Polars is more memory-efficient. A possible explanation is temporary buffers from parallel CSV parsing and from the sort, but this was not tested. Because each value comes from one run, the memory results should be treated as indicative.

7. Scalability Testing

To observe how performance changes with data volume, random samples containing 10%, 25%, 50% and 100% of the rows (fixed seed 42) were created, giving 1.09 million, 2.73 million, 5.45 million and 10.91 million rows. The filter, groupby, join, sort and transform operations were timed at each size

(median of 5 runs). The 10% Polars sample contains one row fewer than the Pandas sample (1,090,685 vs 1,090,686) because the two libraries round the sample size differently.

Table 8. Median time in seconds, Pandas / Polars (speedup), by number of rows

Operation	1.09M rows	2.73M rows	5.45M rows	10.91M rows
filter	0.007 / 0.005 (1.3x)	0.020 / 0.012 (1.6x)	0.033 / 0.024 (1.4x)	0.064 / 0.045 (1.4x)
groupby	0.035 / 0.015 (2.3x)	0.083 / 0.037 (2.3x)	0.162 / 0.074 (2.2x)	0.318 / 0.146 (2.2x)
join	0.013 / 0.007 (1.9x)	0.031 / 0.014 (2.2x)	0.061 / 0.033 (1.8x)	0.123 / 0.052 (2.4x)
sort	0.125 / 0.084 (1.5x)	0.526 / 0.237 (2.2x)	1.545 / 0.539 (2.9x)	3.358 / 1.130 (3.0x)
transform	0.007 / 0.006 (1.3x)	0.019 / 0.013 (1.4x)	0.034 / 0.026 (1.3x)	0.068 / 0.053 (1.3x)

Both libraries scale approximately linearly for filter, groupby, join and transform, and the gap between them stays roughly constant. Sort behaves differently: from 1.09 million to 10.91 million rows (10 times more data), Pandas sort time increases by about 27 times (0.125 s to 3.358 s), while Polars increases by about 13 times (0.084 s to 1.130 s). The Polars speedup for sort therefore grows from 1.5x to about 3.0x as the data grows.

8. Results and Visualization

Equivalence of results. Before the timings were compared, the outputs of both libraries were checked for equivalence: the number of rows returned by the filter and the join, and the groupby aggregates (compared with a tolerance of 1e-6).

Graphs. Figure 1 compares the execution time of every operation on the full dataset (log scale), Figure 2 compares memory use, and Figure 3 shows how execution time and the Polars speedup change with dataset size.

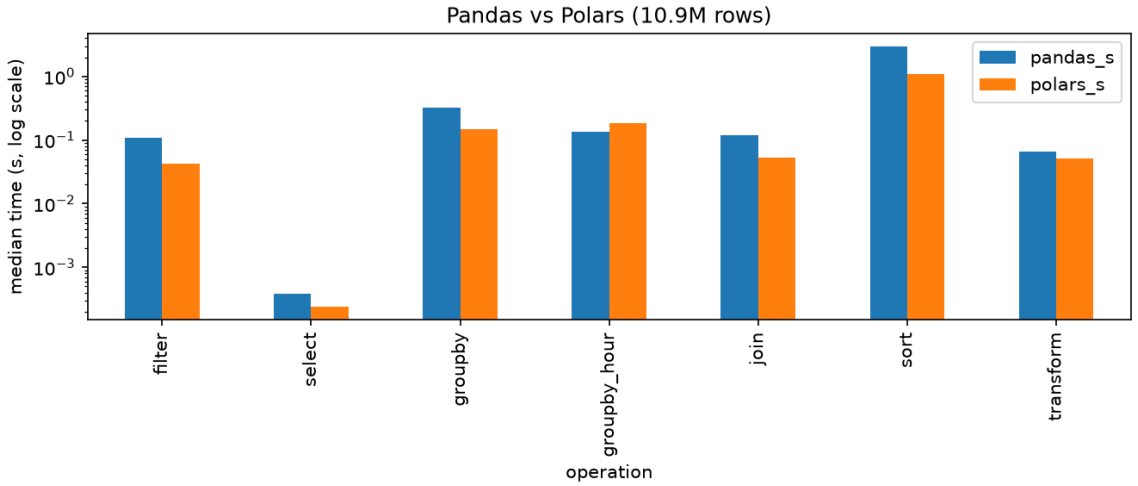


Figure 1. Median execution time per operation (log scale)

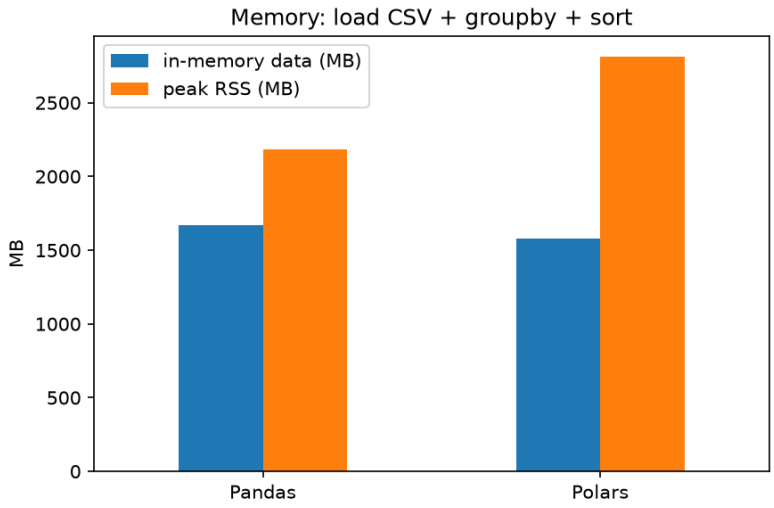


Figure 2. Memory comparison

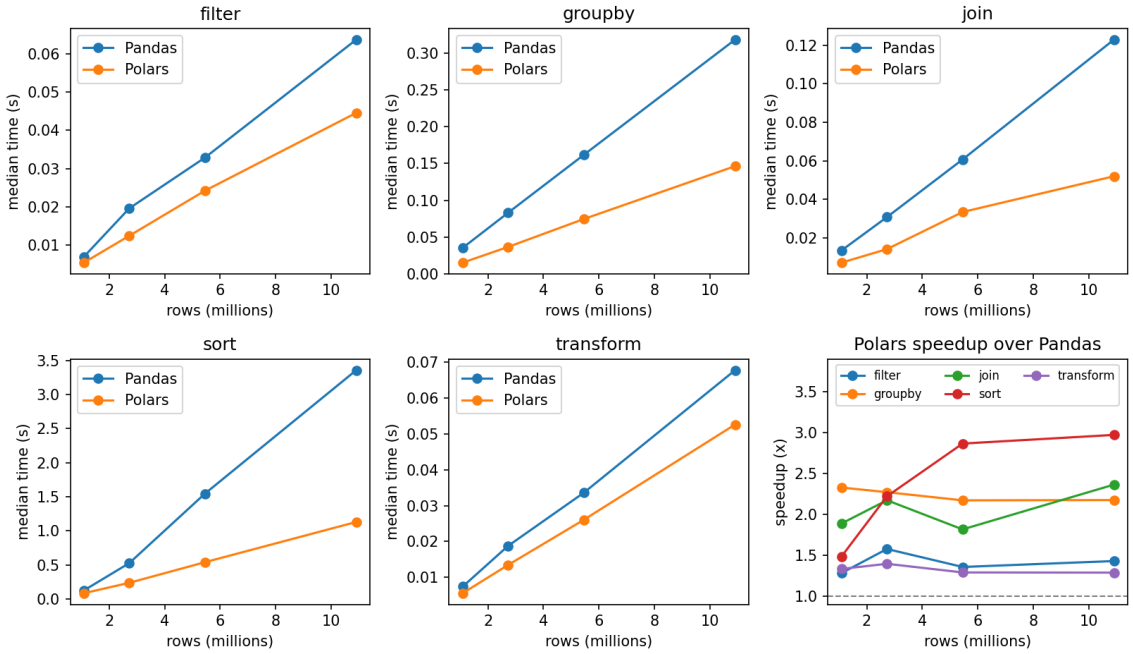


Figure 3. Execution time against dataset size, and Polars speedup over Pandas

Table 9 summarizes the main findings from the tables and graphs above.

Table 9. Summary of findings

Aspect	Finding
Execution time (10.9M rows)	Polars faster for filter, groupby, join, sort and transform (about 1.3x to 2.7x); Pandas faster for grouping by hour (about 1.4x); column selection is instant in both
CSV loading	Similar: Polars about 1.2x faster
Memory	Loaded data size similar (Polars about 5% smaller); peak memory about 29% higher for Polars
Scalability	Both scale roughly linearly; Polars speedup is stable for groupby, join, filter and transform, and grows for sort (1.5x to 3.0x)

9. Analysis and Conclusion

Observations. Polars was faster than Pandas on five of the six comparable operations on the full dataset: filter, groupby, join, sort and transform, by factors between about 1.3x and 2.7x. The largest advantage was for sort and the smallest for transform. This is consistent with Polars using several CPU cores for these operations while most Pandas operations use one, although core usage was not measured directly in this study. Grouping by the hour of a datetime column was about 1.4x faster in Pandas in every run, and column selection was effectively instant in both. For small samples the absolute differences are only a few milliseconds, so the choice of library matters little for small data.

Speedups were largely stable as the data grew, except for sort, where the advantage of Polars increased with size. The difference in CSV loading time was small, so reading a large text file costs similar time in either library; Polars would likely show a larger advantage with columnar formats such as Parquet, but this was not tested. In memory use Polars did not show an advantage in this eager-mode test, and the lazy API, which can avoid loading unused columns, was not evaluated. The Polars results also depended on the memory layout after loading: consolidating chunks improved groupby and sort substantially.

Strengths and limitations. Table 10 summarizes the strengths and limitations of each library as observed in this study and in their design.

Table 10. Strengths and limitations of each library

	Strengths	Limitations
Pandas	Mature, very large ecosystem; familiar, concise syntax; works directly with most data-science libraries; fast for small data and some operations (e.g. hourly groupby here)	Mostly single-threaded; slower on large sorts, joins and aggregations; scaling is limited by available memory
Polars	Faster on most heavy operations (1.3x to 3x here); multi-threaded; consistent expression API; lazy mode with query optimization	Newer, with a smaller ecosystem; more verbose expressions; performance can depend on chunk layout; higher peak memory in this test

Recommended use cases.

- **Pandas** is a good choice for exploratory analysis on small to medium datasets, for teaching, for code that relies on the scikit-learn or plotting ecosystem, and for existing codebases that already use it.
- **Polars** is a good choice for data pipelines and analysis on millions of rows, for sort-, join- and aggregation-heavy workloads, and on multi-core machines. It also offers a lazy API for larger queries, which was not tested here.
- The two can be combined: heavy processing can be done in Polars, and the result converted with `.to_pandas()` for libraries that expect a Pandas DataFrame.

Limitations of this study.

- All measurements come from one machine and one dataset (a single month of taxi data), using only the eager APIs.
- Memory was measured once per library, and some timings (notably Pandas filter and sort) varied between runs.
- The causes of the Pandas filter variation, the higher Polars peak memory and the Pandas advantage in hourly grouping were not investigated, and the cost of `rechunk()` is not included in the Polars timings.

Conclusion. On a 10.9 million row, 1.7 GB taxi dataset, Polars was faster than Pandas for filter, groupby, join, sort and transform, with speedups of roughly 1.3x to 3x, and its advantage on sorting grew with data size. Pandas was faster for grouping by hour, column selection was instant in both, and CSV loading time was similar. In memory use, Polars did not outperform Pandas in this eager-mode test, since its peak memory was higher. Pandas offers a shorter, more familiar syntax and a larger ecosystem, while Polars offers better performance on larger workloads.

Overall, Polars is the better choice when processing speed on large data matters, and Pandas remains a strong default for smaller datasets and ecosystem compatibility. These conclusions apply to the dataset, hardware and eager APIs tested here.